06. Mieux diagnostiquer les erreurs dans `should_fail`

Adaptation de base

Dans sa forme de base, `should_fail` permet bien de vérifier qu'une commande échoue, mais il n'aide pas toujours assez quand l'exception levée n'est pas celle attendue.

Il est donc utile de l'adapter pour :

- mieux afficher la raison d'une exception non attendue ;
- éventuellement afficher aussi l'exception attendue capturée en cas de succès, pour faciliter le diagnostic.

```
create or replace function should_fail(
    sql_cmd text,
    -- voir https://www.postgresql.org/docs/current/errcodes-appendix.html
pour les noms des erreurs
    errname text default 'raise_exception'
)
    returns void as
$$
begin
    begin
        execute sql_cmd;

        /*
        Forcer l'exécution des éventuels triggers différés pour capturer les
erreurs immédiatement

        Un trigger DEFERRED ne s'exécute qu'à la fin de la transaction, et
donc pas immédiatement.
        Dès lors, aucune exception n'est levée immédiatement ici => le test
échoue.
        Pour pouvoir tester un trigger DEFERRED ici, il faut forcer son
exécution avant la fin de la transaction.
        Cette commande change le mode pour les reste de la transaction. Le
mode revient automatiquement à DEFERRED
        à la fin de la transaction.
        */
        execute 'set constraints all immediate';

        raise exception using errcode = 'P0004', message = 'Test failed:
command should have raised an exception';
    exception
        when others then
            if SQLSTATE = get_sqlstate(errname) then
```

```
                -- Exception attendue, OK
                null;
                raise notice 'Expected exception raised: % - %', SQLSTATE,
SQLERRM;
            else
                raise exception using errcode = SQLSTATE, message =
'Unexpected error code: ' || SQLSTATE;
                raise exception using errcode = SQLSTATE, message =
format('Unexpected error code: %s - %s', SQLSTATE, SQLERRM);
            end if;
    end;
end;
$$ language plpgsql;
```

Pour ceux qui veulent aller plus loin

Malheureusement, dans la pratique, les `raise notice` ne sont pas visibles de manière exploitable dans la console **Run** d'une configuration IntelliJ (contrairement aux `raise exception` qui eux le sont).

Autrement dit :

- les `raise notice` écrits directement dans le script peuvent sembler visibles parce qu'IntelliJ affiche le texte du script ;
- en revanche, les `raise notice` exécutés à l'intérieur d'une fonction comme `should_fail` ne remontent pas forcément dans cette console.

Donc le `raise notice` reste utile, mais il ne faut pas compter sur IntelliJ pour bien l'afficher dans ce mode d'exécution.

On peut pallier cela en activant les logs PostgreSQL. Ainsi, même si IntelliJ n'affiche pas correctement les `raise notice`, PostgreSQL les écrira dans ses fichiers de log, ce qui permet ensuite de les consulter.

Pour cela, le plus simple est de configurer PostgreSQL juste après `initdb`, avant le premier démarrage, en ajoutant les paramètres directement dans `postgresql.conf`.

scripts/pg-init.sh

```
#!/bin/bash

cd "$(dirname "$0")"
echo "current directory: $(pwd)"

source env.sh

if [ -f ../data/postmaster.pid ]; then
   pg_ctl -D ../data stop
fi
```

```
if [[ "$(uname -s)" == MINGW* || "$(uname -s)" == MSYS* ]]; then
  taskkill //F //IM postgres.exe >/dev/null 2>&1
  taskkill //F //IM postgrest.exe >/dev/null 2>&1
else
  killall postgres
  killall postgrest
fi

if [ -d ../data ]; then
   rm -rf ../data
fi

mkdir ../data
initdb -D ../data --locale=C --encoding=UTF8

cat >> ../data/postgresql.conf <<'EOF'
logging_collector = on
log_min_messages = notice
EOF

pg_ctl -D ../data start

sleep 3

createuser -s postgres

psql -U postgres -c "
create role authenticator noinherit login password 'mysecretpassword';
create role anon nologin;
create role authenticated nologin;
"
psql -U postgres -c "alter user postgres password 'dummy123'"

psql -U postgres -c "ALTER DATABASE template0 SET timezone =
'Europe/Brussels';"

createdb -T template0 "$DB_NAME"

pg_ctl stop -D ../data -w -m fast
```

Il est plus simple de faire cela dans `pg-init.sh` plutôt qu'avec `ALTER SYSTEM` parce que :

- `ALTER SYSTEM` nécessite que PostgreSQL soit déjà démarré ;
- `logging_collector = on` ne prend effet qu'après redémarrage ;
- en écrivant directement dans `postgresql.conf` avant le premier `start`, le serveur démarre immédiatement avec les bons réglages.

C'est donc plus simple et plus fiable dans un script d'initialisation.

Pour que `pg-init.sh` se ré-exécute, vous devez arrêter le serveur PostgreSQL, supprimer le dossier data et relancer le serveur.

Avec la configuration ci-dessus, les logs seront créés dans :

```
../data/log/
```

Vous pourrez y retrouver les `NOTICE` et `ERROR` produits par PostgreSQL, y compris ceux émis depuis `should_fail`.